

Compiler Infrastructure / Language Tooling Engineer with professional LLVM 22 and C#.NET static-analysis experience. I build UniversalToolchain around deterministic compiler-pipeline composition and also work on LLVM transformations, program analysis, and backend/codegen validation.

2022: Compilers / VMs · 2023: CIL + native x86-64 · 2024–2025: AST / IR / backends → toolchain · 2026 → : Professional LLVM + static analysis

PROFESSIONAL EXPERIENCE

SKILLS

- 2026 — present

ISP RAS — Static Analysis Engineer
 - Contribute to SharpChecker, ISP RAS's C#.NET static-analysis platform.
 - Current work includes symbolic-computation and symbolic-execution tasks in C#.NET program analysis.
- July — August 2026

MCST — Compiler Engineering Intern
 - Implemented a LICM pass for LLVM 22 in C++; hoisting conditions check loop structure, memory access, side effects, and speculative safety.
 - Built a Python harness comparing program behavior before/after optimization and checking transform/no-transform cases.

Additional product experience — Diploma.school — client development: jQuery dropdown interactions, responsive mobile/desktop navigation, and UI regression fixes; confirmed no later than January 2025.

PROJECTS

- UniversalToolchain**

Language features resolve dependencies and conflicts, select capability/runtime providers, obtain deterministic contribution ordering, and build typed artifact routes before runtime materialization.

Current manifest: 1,324/1,324 tests; dedicated checks cover interpreter/CIL parity and configuration integrity before execution.
- LLVM Interprocedural Global IV Optimization**

An LLVM 22 pass localizes supported induction-like globals after interprocedural effect analysis, affine evolution, and a separate applicability check; APInt models fixed-width wraparound.

29 positive/negative regression cases cover LLVM Verifier checks, structural expectations, idempotence, and observable before/after behavior.
- x86-64 Compiler Backend & Register Allocation**

SSA/CFG validation → liveness/interference → register allocation/spills → phi lowering → SysV x86-64 emission.

An independent assignment verifier plus native-vs-interpreter differential execution; tests cover branches, loops, spills, and parallel phi moves; a separate public native-codegen lineage starts with Visk (2023) and Wist4 (2024).

- Compiler Infrastructure**

dependency/conflict resolution, component/provider selection, deterministic pass ordering, typed artifact routing, pre-execution checks
- Compiler / IR**

C++23, LLVM 22/IR, CFG/SSA, loop and interprocedural analysis, transformation applicability
- Backends / Runtime**

.NET, CIL/MSIL, native x86-64, SysV ABI, stack frames/calling conventions, register allocation, interpreter/runtime composition
- Verification**

regression suites, LLVM Verifier, interpreter/backend parity, differential execution

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

- LangDev'26 — Build the Language, Then Make the Abstractions Disappear**

Accepted talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.
- UniversalToolchain — Baltic Science and Engineering Competition**

First Degree Diploma and Grand Prize "Perfection as Hope" for UniversalToolchain.
- MEPhI Junior**

Overall winner; First Degree Diploma, 96/100 for UniversalToolchain.
- Moscow / remote